

Voice Scribe: End-to-End Implementation Plan

This implementation plan outlines the steps required to build, test, and deploy the complete software pipeline for **Voice Scribe: Voice-Operated Autonomous Physical Writing Machine**, converting spoken words into physical pen-on-paper handwriting based on the project technical specifications.

Goal Description

The objective is to implement a robust, production-grade Python software suite that interfaces with the existing physical hardware (Arduino Uno + CNC Shield V3 + A4988 steppers + SG90 pen servo on GT2 Cartesian gantry).

The software will:

1. Capture voice via microphone and transcribe it with acoustic noise calibration.
 2. Provide a two-stage staging buffer preventing ink deposition until explicitly confirmed by voice ("OK") or keyboard.
 3. Compile text into authentic single-stroke centerline vectors using the Hershey Simplex font table (preventing hollow/double-line fonts).
 4. Perform automatic word-wrapping and line-discipline against standard A4 paper dimensions.
 5. Stream G-code commands to GRBL via a strict synchronous ping-pong protocol (send \$\\rightarrow\$ wait for ok \$\\rightarrow\$ next) with servo settling dwell (G4 P0.35) and DTR reboot protection.
 6. Provide dedicated diagnostic test suites to safely test motors, servo angles, and audio capture before running live dictation.
-

User Review Required

IMPORTANT

COM Port Selection: The document specifies default COM8 . When running on your PC, we will implement auto-detection of connected Arduino COM ports, but you can override this in config.json .

Speech-to-Text Mode:

- **Option A (Default / Fast Setup):** Google Speech Recognition via SpeechRecognition library. Works out-of-the-box, very high accuracy, but requires internet connection.
- **Option B (Exam Hall Compliant / Offline):** Local Vosk engine. 100% offline, requires downloading a lightweight language model (~40MB).

We propose supporting both with a toggle in config.json .

Open Questions

NOTE

1. Is your Arduino Uno currently plugged in, and do you know which COM port Windows assigned to it (e.g. COM3, COM4, COM8)?
2. Would you prefer the initial launch to default to Google Speech (online) or Vosk (offline)?

Architecture & Data Flow

```

flowchart TD
    subgraph Audio_Layer [Audio Layer]
        MIC[USB Headset / Microphone] --> VAD[Noise Calibration & Capture]
        VAD --> ASR[Speech-to-Text Engine\nGoogle ASR / Offline Vosk]
    end

    subgraph State_Machine [State Machine]
        ASR --> BUF[Staged Preview Buffer]
        BUF --> CMD[Command Classifier]
        CMD -- "OK / CONFIRM" --> COMMIT[Commit Text to Write]
        CMD -- "NEW LINE" --> NL[Advance Carriage Down Y]
        CMD -- "CLEAR / CANCEL" --> CLR[Discard Buffer]
        CMD -- New Sentence --> BUF
    end

    subgraph Vector_Layout_Engine [Vector & Layout Engine]
        COMMIT --> HERSHEY[Hershey Simplex Font Compiler]
        HERSHEY --> WRAP[Word Boundary & Margin Checker]
        WRAP --> GCODE[G-code Sequence Generator\nG0, G1, G4, M03]
    end

    subgraph Hardware_Motion_Stream [Hardware Motion Streamer]
        GCODE --> STREAM[GrblMotionStreamer\nPing-Pong Flow Control]
        STREAM -- "USB Serial 115200 Baud" --> ARD[Arduino Uno + CNC Shield V3]
        ARD --> STEP[NEMA 17 Steppers X/Y]
        ARD --> SERVO[SG90 Micro Servo Pen Lift]
    end

```

Proposed Changes

We will build a clean, modular Python codebase in the current workspace directory

`C:\Users\hp\Desktop\VOICE TRANSCRIBE MACHINE`.

```

VOICE TRANSCRIBE MACHINE/
├─ config.json           # Centralized hardware, geometry & voice configuration
├─ requirements.txt      # Python package dependencies
├─ grbl_controller.py    # Hardware communication & ping-pong motion streamer
├─ hershey_fonts.py      # Single-stroke Hershey vector glyph lookup table
├─ vector_compiler.py    # Kerning, word-wrapping, and trajectory compiler
├─ voice_engine.py       # Microphone capture, noise reduction, and ASR
├─ voice_scribe_engine.py # Main production orchestrator loop
├─ test_hardware.py      # Diagnostic tool for COM port, steppers, and servo
└─ test_audio.py         # Diagnostic tool for microphone & transcription check

```

Configuration & Dependencies

[NEW] `config.json`

Stores all machine parameters cleanly outside the code so adjustments (feed rates, margins, COM port, servo angles) can be tuned without editing Python scripts.

```

{
  "serial": {
    "port": "COM8",
    "auto_detect": true,
    "baud_rate": 115200,
    "boot_delay_seconds": 2.5,
    "timeout": 2.0
  },
  "motion": {
    "cmd_pen_up": "M03 S90",
    "cmd_pen_down": "M03 S0",
    "servo_dwell": 0.35,
    "rapid_feed": 2500,
    "draw_feed": 1000
  },
  "geometry": {
    "page_width": 170.0,
    "page_height": 290.0,
    "margin_left": 15.0,
    "start_y": 270.0,
    "line_spacing": 9.5,
    "font_scale": 0.28
  },
  "speech": {
    "engine": "google",
    "ambient_duration": 1.0,
    "phrase_timeout": 8.0
  }
}

```

[NEW] `requirements.txt`

Specifies necessary dependencies:

```
pyserial ≥ 3.5
SpeechRecognition ≥ 3.10.0
pyaudio ≥ 0.2.14
```

Hardware Communication Layer

[NEW] `grbl_controller.py`

Encapsulates `GrblMotionStreamer`:

- Auto-detects Arduino CH340 / Uno COM port if `auto_detect` is enabled.
- Connects at 115200 baud.
- Applies the mandatory **2.5s DTR delay** to allow the bootloader to finish resetting.
- Dispatches safety alarm unlock (`$X`), metric positioning (`G21`), and absolute mode (`G90`).
- Implements strict **synchronous ping-pong flow control** (`send_blocking`) with line buffering and timeout safety.
- Handles pen actuation:
- `pen_up()` : dispatches `M03 S90` followed by `G4 P0.35` dwell.
- `pen_down()` : dispatches `M03 S0` followed by `G4 P0.35` dwell.
- Linear and rapid displacements: `rapid_move(x, y)` (`G0`), `linear_draw(x, y)` (`G1 ... F1000`).
- Safe exit handler to raise pen and park carriage.

Vector Font & Layout Engine

[NEW] `hershey_fonts.py`

Contains the Hershey Simplex single-stroke vector database:

- Uppercase letters `A-Z`.
- Numbers `0-9`.
- Punctuation symbols (`.`, `,`, `-`, `:`, `?`, `!`, `'`, `"`, `/`, `(`, `)`, space).
- Case-mapping: maps lowercase letters to clean single-stroke glyphs or normalized capitals so dictation never breaks on mixed-case words.
- Each glyph includes its advance width and stroke coordinate tuples.

[NEW] `vector_compiler.py`

Encapsulates `VectorLayoutCompiler`:

- Tracks `cursor_x` and `cursor_y`.
- Origin initialization to `(margin_left, start_y) = (15.0, 270.0)`.
- `compute_word_width(word)` : calculates exact physical millimeter width with font scaling and kerning.
- Auto word-wrapping: checks if `cursor_x + word_width > (page_width - margin_left)` and triggers automatic carriage return and line feed.

- Line advancement: drops `cursor_y` by `line_spacing` (9.5mm), with bottom boundary guard ($Y < 15\text{mm}$).
 - Trajectory generator: iterates strokes, executing rapid move $\rightarrow$ pen down $\rightarrow$ draw segment lines $\rightarrow$ pen up.
-

Speech & Interaction State Machine

[NEW] `voice_engine.py`

Handles audio capture and NLP parsing:

- Initializes microphone with automatic ambient noise profile adjustment.
- Captures audio chunks with phrase timeouts.
- Transcribes using selected engine (Google ASR / Vosk).
- Identifies control triggers:
 - **Newline:** `"new line"`, `"newline"`, `"enter"`, `"next line"`
 - **Confirm/Execute:** `"ok"`, `"okay"`, `"confirm"`, `"write it"`
 - **Discard:** `"clear"`, `"cancel"`, `"discard"`, `"erase"`

[NEW] `voice_scribe_engine.py`

The unified main application:

- Loads `config.json`.
 - Instantiates `GrblMotionStreamer`, `VectorLayoutCompiler`, and `VoiceEngine`.
 - Displays a clean, readable terminal UI showing:
 - System status and connection banner.
 - Active staged preview buffer.
 - Prompt commands.
 - Handles keyboard interrupts (`Ctrl+C`) cleanly by lifting the pen, parking the carriage, and releasing the serial port.
-

Diagnostic & Hardware Testing Tools

[NEW] `test_hardware.py`

A standalone, non-voice interactive hardware test utility:

- Scans and lists all active COM ports.
- Verifies GRBL connection and prints firmware version.
- **Interactive Menu:**
 1. Toggle Pen Up (`S90`) / Pen Down (`S0`) to visually verify servo arm height.
 2. Jog X and Y axes (+10mm, -10mm).
 3. Draw a 20mm $\times$ 20mm test calibration square.
 4. Write a sample single-stroke test word (e.g. `"TEST"`).

5. Read and display GRBL configuration (`$$`).

[NEW] `test_audio.py`

A standalone audio test utility:

- Lists all connected audio input devices (helps verify the USB headset is selected).
- Records a 3-second sample and runs transcription to verify microphone sensitivity without moving the robot.

Verification Plan

Phase 1: Environment & Audio Verification

1. Virtual Environment Setup:

```
py -3 -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

2. Audio Input Test:

```
.\venv\Scripts\python test_audio.py
```

Expected Result: Detects USB microphone, records test phrase, and outputs accurate transcript.

Phase 2: Hardware & Motion Verification

1. COM Port & GRBL Communication:

```
.\venv\Scripts\python test_hardware.py
```

Expected Result: Connects to Arduino, unlocks `$X`, reads GRBL banner.

2. Servo Cam Test:

- Test Option 1 (Pen UP / DOWN) in `test_hardware.py`.
- Verify pen clears paper by 2–4mm when UP and contacts paper with spring pressure when DOWN.

3. Dimensional Calibration Test:

- Draw a 20mm test square using `test_hardware.py`.
- Measure with a physical ruler to verify steps/mm resolution ($100 \times \text{steps/mm}$).

Phase 3: Full End-to-End Dictation Test

1. Launch Orchestrator:

```
.\venv\Scripts\python voice_scribe_engine.py
```

2. Test Dictation Flow:

- Speak: "The quick brown fox" $\rightarrow$ verify text stages in console; robot does not move.
- Say: "OK" $\rightarrow$ verify pen drops and accurately writes the sentence in physical ink with proper kerning and zero streaks.
- Say: "NEW LINE" $\rightarrow$ verify pen lifts and moves down by 9.5mm.
- Speak: "CLEAR" test $\rightarrow$ dictate phrase, say "CANCEL", verify buffer is erased without drawing.